CPC — Lecture 4

The 100 Prisoners proof, "Pearls", Duplicate element in an array

Competitive Programming and Contests — Monday 28/09/2026

Sources. Recording `L_04.MP4` (1h32m): Teams transcript (`L_04-en-US.docx`) + frames of the professor's iPad notes (the handwritten images are *his* board). Prof.'s notes of last year on the same topics: `Pearls_2025.pdf` (in this folder), used for the parts marked [extra] (the PDF is now in `Lessons/L5/`). Your `notes.md` and your `my_sol` were also used.

Where the lecture ended. The prof stopped at §3.6 ("Does random access help? Destroying A"): the bug of that solution is left as homework. The lecture continues in L5 (30/09): `Lessons/L5/L5.pdf`, where the prof also says how to fix the bug (Quicksort partitioning).

Code: `Lessons/L4/src/claude.rs` (your `lib.rs` is untouched apart from the line `pub mod claude;`) — `cargo test -p l4`.

Contents

1 The 100 Prisoners puzzle: the proof **1**
- 1.1 Recap and strategy . . . 1
- 1.2 Counting . . . 2

2 "Pearls" and the spirit of the course **3**

3 Duplicate element in an array **3**
- 3.1 Problem . . . 3
- 3.2 Solution 1: hash set . . . 4
- 3.3 Solution 2: direct access table . . . 4
- 3.4 Constant extra space? . . . 4
- 3.5 Solution 3: $\Theta(\log n)$ passes, reconstruct e bit by bit . . . 4
- 3.6 Does random access help? Destroying A . . . 5
- 3.7 [extra] Your solution `my_sol` . . . 6

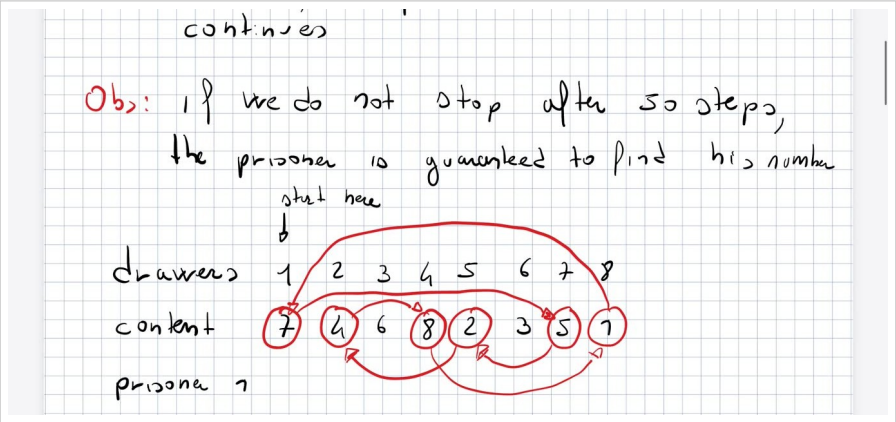
1 The 100 Prisoners puzzle: the proof

1.1 Recap and strategy

100 prisoners, 100 drawers with a permutation of $1..100$, each opens at most 50 drawers; they are freed only if *all* find their number. Random strategy: $P = 1/2^{100}$. The strategy found in L3:

- the **drawers are numbered too**: if the numbers inside are random, any numbering works (e.g. row-wise); otherwise the prisoners assign the numbers to the drawers *uniformly at random* — this is where randomness comes in;
- prisoner i opens drawer i, finds d_i; if $d_i = i$ he wins, otherwise he opens drawer d_i and continues.

Observation: if we do not stop after 50 steps, the prisoner is guaranteed to find his number (it is a permutation: he walks along a cycle and comes back to the start).



Example with 8 drawers, contents $7, 4, 6, 8, 2, 3, 5, 1$: prisoner 1 visits $1 \to 7 \to 5 \to 2 \to 4 \to 8$ and finds 1 in drawer 8: he is closing the cycle. He wins iff his cycle has at most half of the drawers, and then everybody in that cycle wins with him. [extra] Here the cycles are $(1\,7\,5\,2\,4\,8)$ and $(3\,6)$: with 4 openings each, the six prisoners of the long cycle lose, so all lose.

$$P(\text{all win}) = P(\text{a random permutation has no cycle longer than } 50).$$

1.2 Counting

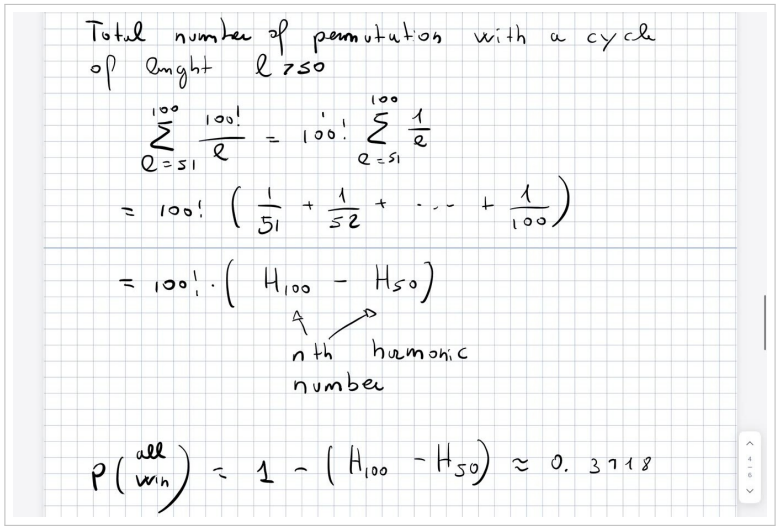
Count the permutations of 100 elements with a cycle of length $\ell > 50$:

- since $\ell > 50$ there is **only one** such cycle (it covers more than half of the elements) $\Rightarrow$ no double counting;
- choose the elements of the cycle: $\binom{100}{\ell}$;
- order them in the cycle: $\ell!$ orderings, but a cycle has no beginning, so each cycle is counted ℓ times (e.g. $\ell = 3$, elements $3, 5, 42$: $3\,5\,42$, $5\,42\,3$, $42\,3\,5$ are the same cycle) $\Rightarrow (\ell-1)!$;
- order the remaining $100 - \ell$ elements freely: $(100-\ell)!$.

$$\binom{100}{\ell}(\ell-1)!\,(100-\ell)! = \frac{100!}{\ell}, \qquad \sum_{\ell=51}^{100} \frac{100!}{\ell} = 100!\left(\frac{1}{51} + \cdots + \frac{1}{100}\right) = 100!\,(H_{100} - H_{50})$$

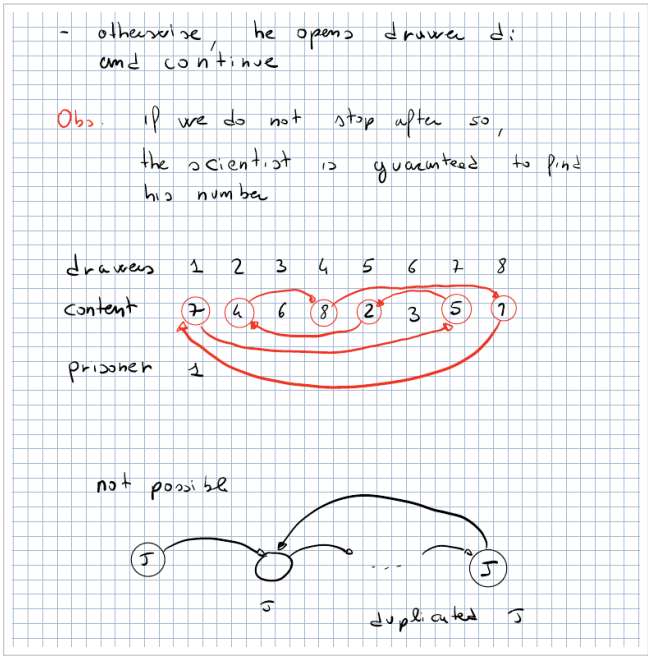
with $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$ the n-th harmonic number. Dividing by the 100! permutations:

$P(\text{all win}) = 1 - (H_{100} - H_{50}) \approx 0.3118$ (vs. $\approx 10^{-30}$ with the random strategy). [extra] For $n \to \infty$ it tends to $1 - \ln 2 \approx 0.307$; simulation in `Lessons/L3` (`--bin prisoners`): 0.313.



Why it is exciting: a problem that looks impossible to improve has a simple strategy that is $\sim 10^{29}$ times better, and the reason is also simple: the strategy *couples* the outcomes of the prisoners in the same cycle, and short cycles are not that unlikely.

[extra] **From last year's notes.** (i) A scientist does not need to choose the drawers beforehand: the strategy is *adaptive*. (ii) Why following the numbers always closes a cycle: in a permutation every drawer is pointed to by exactly one number, so a "ρ" shape (two arrows entering the same node) is impossible — it would require a *duplicated* number. Keep this picture in mind for the next problem.



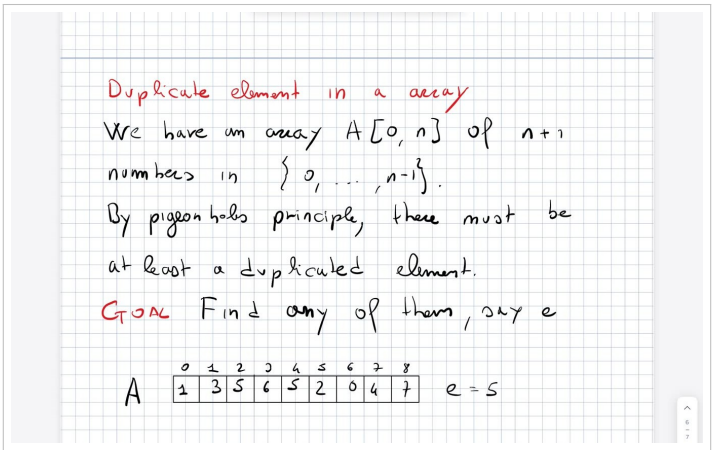
Prof.'s notes of **last year**, `Lessons/L5/Pearls_2025.pdf` p. 3

2 "Pearls" and the spirit of the course

The current series of lectures (started at the end of L3) are **pearls**: small, apparently simple problems with super cute, clever solutions; what we try to solve looks impossible, but an elegant and simple solution exists. The problems are connected: permutations and cycles from the puzzle will be useful for the next ones. In general the course presents *many small problems* per lecture; the focus is on the *techniques*.

3 Duplicate element in an array

3.1 Problem



Given $A[0..n]$ with $n+1$ numbers in $\{0,\dots,n-1\}$, by the pigeonhole principle there is at least one duplicate. **Find any** duplicate e. Example: $A = [1,3,5,6,5,2,0,4,7]$ ($n = 8$), $e = 5$. (The prof noted he used 0-based indexing here, unlike the other problems, without a particular reason.)

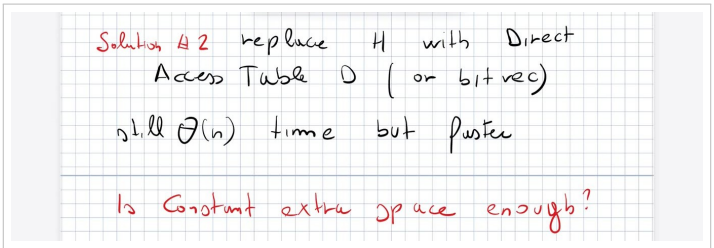
3.2 Solution 1: hash set

Scan A, add each element to a hash set H, stop as soon as the current element is already in H. $\Theta(n)$ time **with high probability** (hashing is randomized; an unlucky run is slower, and even a good hash function has some collisions, i.e. more than n memory accesses), and $\Theta(n)$ extra space: to guarantee $O(1)$ operations w.h.p. the table must be 2–4$\times$ the number of elements, it cannot be asymptotically smaller than n (answer to a question in class).

A student's idea that does not work. Sum the array and subtract $0+1+\cdots+(n-1)$: this works only if A is a permutation plus one duplicate, but here any multiset is allowed (e.g. $5,5,5,5,\dots$).

3.3 Solution 2: direct access table

Values are in $\{0,\dots,n-1\}$: replace H with a **direct access table** (a bit vector D of n bits, all 0; set $D[x] = 1$ when x is seen). Still $\Theta(n)$ time, but deterministic and *faster*: no hashing, direct access, the CPU can predict and prefetch.



Whenever you reach for a hash map (dictionary in Python, map in C++/Rust), ask: *do I really need it, or does my data allow a direct access table?* Hashing is usually taught the other way round: start from a direct access table as large as the largest value, then hash because the table is too big. If the table is small enough, there is no reason to pay for hashing and collisions.

3.4 Constant extra space?

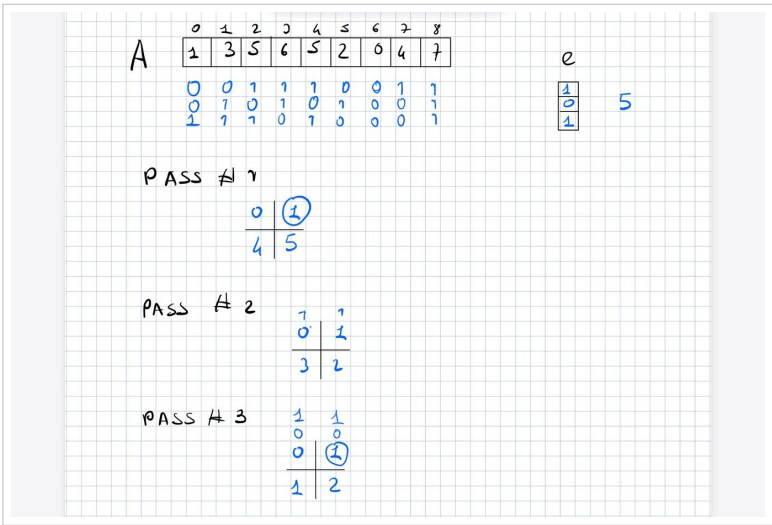
It seems impossible: we need to remember the elements seen so far. A student proposed to multiply, for each element x, an accumulator by the x-th prime, and factorize at the end: but the number can be as large as $\approx n!$, i.e. $\Theta(n \log n)$ bits — as much as the array itself. (If you can reconstruct the content of the array, you must use at least the space needed to represent it.) "Even incorrect solutions are nice."

3.5 Solution 3: $\Theta(\log n)$ passes, reconstruct e bit by bit

Assume n is a power of 2. Write the elements in binary and find the bits of e from the most significant one, with one scan per bit.

- **Pass 1**: count zeros and ones of the most significant bit. Without the extra element we would have exactly half and half (4 numbers are < 4, 4 are ≥ 4); the duplicate unbalances them. Example: 4 zeros, 5 ones $\Rightarrow$ first bit of e is 1.
- **Wrong continuation**: repeat the same count on the 2nd and 3rd bit over the whole array. Counterexample built in class: put a 3 in the first position, $A = [3,3,5,6,5,2,0,4,7]$: the per-bit majority gives $111 = 7$, which is not a duplicate.

- **Correct**: at each pass count only the elements that start with the prefix found so far. Why: the elements starting with 1, minus 4, form a smaller instance of the same problem (5 elements with values in $0..3 \Rightarrow$ a duplicate by pigeonhole). Example: pass 2 on $\{5, 6, 5, 4, 7\}$: 3 zeros, 2 ones $\Rightarrow$ 0; pass 3 on $\{5, 5, 4\}$: 1 zero, 2 ones $\Rightarrow$ 1; $e = 101_2 = 5$.



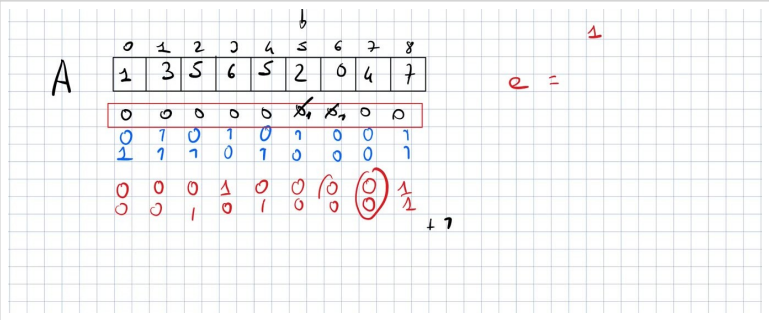
$\Theta(n \log n)$ time, $O(1)$ extra space. It is the **best possible if no random access is allowed** (only left-to-right scans).

[extra] `claude::bit_by_bit` works for any n: it halves the *range of values* $[lo, hi)$ that surely contains a duplicate (if more than $mid - lo$ elements fall in $[lo, mid)$, a duplicate is there); with n a power of 2 it is exactly the bit-by-bit counting. `claude::bits_majority_naive` is the wrong version, with the counterexample as a test.

3.6 Does random access help? Destroying A

Can we get $\Theta(n)$ time *and* $O(1)$ extra space if random access is allowed? The prof first presented a "cheating" approach: we may **destroy** A, reusing its space.

- Idea from a student: do pass 1, so the most significant bit of e is known. Then the most significant bit of every element is no longer needed: n free bits, exactly what Solution 2 needs. Use them as the bit vector: to remember that 5 was seen, set the MSB of $A[5]$ to 1 (the number stored there no longer makes sense, but it was already accounted for). One more pass $\Rightarrow$ linear time.
- **The bug**: in the single pass we must consider only the elements whose MSB was 1 and skip the others; but once the MSBs are overwritten, $1 = 001$ and $5 = 101$ have the same remaining bits and cannot be told apart.
- Attempts in class: negative numbers or a special marker for the elements to skip. Still not enough: with 2 remaining bits we have 4 values and would need $4 + 1$ symbols; e.g. using 00 as the marker collides with $4 = 100$, which would look like a duplicate.



Homework: fix the bug of this solution ($\Theta(n)$ time, $O(1)$ extra space, A can be destroyed). The lecture ended here.

3.7 [extra] Your solution `my_sol`

Your `my_sol` sends each value v to cell v by swapping, and stops when the cell v already contains v. It destroys A, uses $O(1)$ extra space and runs in $\Theta(n)$: every swap puts one value in its final cell for good, so there are at most n swaps overall (the `while` inside the `for` does not make it quadratic). It is a valid answer to "does random access help, if we may destroy A?", different from the approach seen in class. Tests in `claude.rs`: 3000 random arrays checked against the definition, plus an adversarial input of size $2 \cdot 10^5$ (one big cycle). All pass.

Next: L5

The duplicate element continues in L5 (moving every element to its cell, Floyd), followed by the majority element (Boyer–Moore): see `Lessons/L5/L5.pdf`. Exercises for this lecture: `Lessons/L4/practice` (`cargo test -p practice_l4`).